

Цель:

Целью данной лабораторной работы является получение практических навыков реализации и обучения полносвязной нейронной сети с нуля на языке Python с использованием объектно-ориентированного подхода. Вам нужно реализовать алгоритм обратного распространения ошибки (backpropagation) с применением градиентного спуска на минибатчах (mini-batch gradient descent) для обучения сети на задаче бинарной классификации.

Задачи:

1. Изучить теоретические основы алгоритма обратного распространения ошибки и градиентного спуска на минибатчах.
2. Реализовать класс `NeuralNetwork`, инкапсулирующий структуру и методы полносвязной нейронной сети.
3. Реализовать основные функции активации (Sigmoid, ReLU, Tanh) и их производные.
4. Реализовать метод прямого распространения сигнала (`_forward`) через слои сети.
5. Реализовать метод вычисления функции потерь (`_compute_cost`), используя перекрестную энтропию.
6. Реализовать основной метод обратного распространения ошибки (`_backward`) для вычисления градиентов по весам и смещениям.
7. Реализовать метод обновления параметров сети (`_update_parameters`) на основе вычисленных градиентов.
8. Реализовать метод обучения (`fit`), организующий процесс обучения по эпохам с использованием мини-батчей.
9. Реализовать метод предсказания (`predict`) для получения выходных данных сети.
10. Протестировать реализованную нейронную сеть на синтетическом наборе данных `make_moons` и визуализировать результаты.

Теория:

Основные концепции:

1. Полносвязная нейронная сеть: Сеть, где каждый нейрон слоя связан со всеми нейронами предыдущего слоя.

2. Прямое распространение: Вычисление выхода сети путем передачи входных данных через слои с применением весов, смещений и функций активации.
3. Функция потерь: Бинарная перекрестная энтропия, измеряющая расхождение между предсказаниями и истинными значениями.
4. Обратное распространение: Алгоритм вычисления градиентов функции потерь по параметрам сети с использованием цепного правила.
5. Градиентный спуск: Итерационный метод оптимизации, корректирующий параметры сети в направлении, противоположном градиенту.

Описание:

Класс `NeuralNetwork` реализует полносвязную нейронную сеть для бинарной классификации с возможностью настройки архитектуры и функций активации.

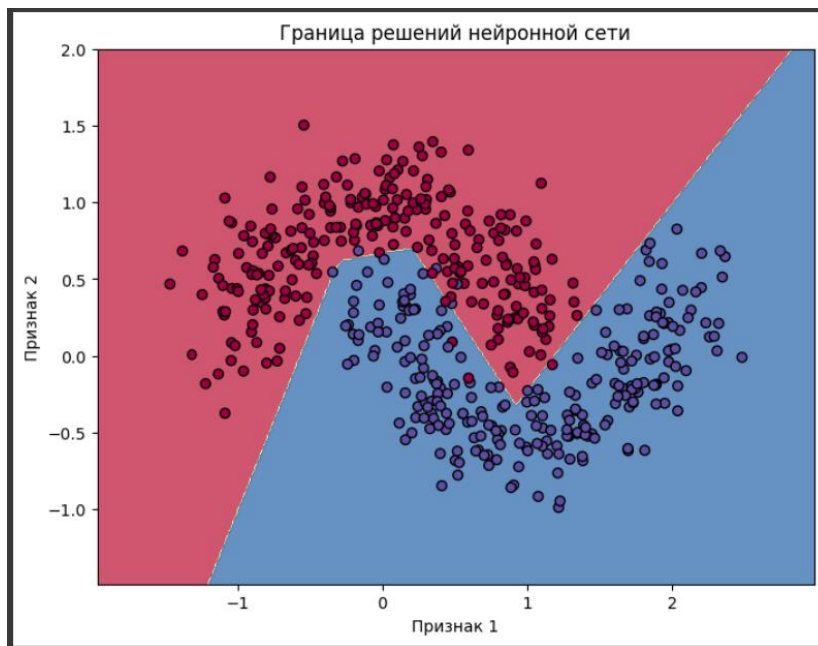
1. `init(self, layer_dims, activation_funcs, learning_rate=0.01)` Инициализация нейронной сети.
2. `_forward(self, X)` Прямое распространение сигнала.
3. `_compute_cost(self, A_last, Y)` Вычисление функции потерь (бинарная перекрестная энтропия).
4. `_backward(self, A_last, Y, cache)` Обратное распространение ошибки.
5. `_update_parameters(self, grads)` Обновление параметров градиентным спуском.
6. `fit(self, X_train, Y_train, epochs, batch_size, print_cost_every=100)` Обучение сети.
7. `predict(self, X)` Предсказание для новых данных.

Результаты тестирования:

График функции потерь.



Визуализация границы решений.



Значение точности (ассурасу) на обучающей выборке.

Точность на обучающем наборе: 98.00%

Анализ результатов и выводы по работе.

В ходе работы была успешно реализована полносвязная нейронная сеть с алгоритмом обратного распространения ошибки. Модель продемонстрировала высокую точность на синтетических данных. Реализация подтвердила теоретические основы работы нейронных сетей и алгоритмов их обучения.

Ссылка:

<https://colab.research.google.com/drive/1cynBKsUObZAHqwVg1rBZDLO0loDtfJvV?usp=sharing>

Контрольные вопросы:

1. Алгоритм прямого распределения

- Принимаем входные данные X

- Для каждого слоя l от 1 до L :

$$Z^l = W^l @ A^{l-1} + b^l$$

$A^l = \text{activation}(Z^l)$

- Сохраняем промежуточные значения Z^l , A^l в кэш

2. Формула бинарной перекрестной энтропии и ее смысл

$$J = -1/m * \sum [y_i * \log(a_i) + (1-y_i) * \log(1-a_i)]$$

Смысл:

- Мера расхождения между предсказанными (a_i) и истинными (y_i) значениями

- Сильно штрафует уверенные, но неправильные предсказания

- m - количество примеров в батче

3. Основная идея алгоритма обратного распространения ошибки.

- Вычисляем ошибку на выходном слое

- Рекурсивно распространяем ее назад через сеть

- Используем цепное правило для вычисления производных

Роль: Позволяет разложить производную сложной функции на произведение производных

4. Как вычисляются градиенты dW^l и db^l для скрытого слоя l ?

$$dZ^l = dA^l * g'(Z^l) \quad \# g' - \text{производная активации}$$

$$dW^l = 1/m * dZ^l @ A^{l-1}.T$$

$$db^l = 1/m * \text{sum}(dZ^l, \text{axis}=1)$$

$$dA^{l-1} = W^l.T @ dZ^l$$

5. Разница между пакетным, стохастическим и мини-батчевым градиентным спуском

Пакетный:

- Размер батча: весь датасет

- Преимущества: Стабильные градиенты

- Недостатки: Медленный, требует много памяти

Стохастический:

- Размер батча: 1 пример

- Преимущества: Быстрый, избегает локальных минимумов
- Недостатки: Шумные градиенты

Мини-батчевый:

- Размер батча: 16-512
- Преимущества: Баланс скорости и стабильности
- Недостатки: Требуется настройка размера батча

6. Зачем нужно перемешивать данные перед каждой эпохой при использовании мини-батчей?

- Предотвращает запоминание порядка примеров
- Обеспечивает репрезентативность мини-батчей
- Помогает избежать цикличности в обучении

7. Какую роль играет скорость обучения (α)?

Роль:

Контролирует размер шага при обновлении параметров.

Проблемы:

Слишком большая $\rightarrow$ Колебания или расходимость

Слишком маленькая $\rightarrow$ Долгое обучение

8. Какие проблемы могут возникнуть при обучении глубоких сетей?

- Исчезающие градиенты
- Взрывающиеся градиенты
- Переобучение
- Мертвые нейроны